INSTITUTO POLITECNICO NACIONAL
ESCUELA SUPERIOR DE COMPUTO

EXPERIMENTO 11.6

"PROCESAMIENTO DE IMÁGENES PARA HUELLAS DACTILARES"

PROCESAMIENTO DIGITAL DE SEÑALES

PROF. PUEBLA LOMAS JAIME HUGO

5CM2

- ALABAT MONTOYA MARIA FERNANDA
- BRIONES CRUZ JUAN CARLOS
- SALAS ORTÍZ ESTEPHANI
- VAZQUEZ RODRIGUEZ ULISES

INSTITUTO POLITÉCNICO NACIONAL

ESCOM

1. OBJETIVO

Crear un programa en lenguaje C que permita el procesamiento de una huella dactilar.

2. INTRODUCCIÓN

La programación al igual que las matemáticas, suele aplicarse para gran diversidad de resolución a problemáticas, así involucrándose en diferentes ámbitos. En este caso, nosotros como estudiantes de la carrera Ing. en Sistemas Computacionales nos vemos a la tarea de aplicar nuestros conocimientos en programación, para el desarrollo y entendimiento de la unidad de aprendizaje en cuestión "Procesamiento Digital de Señales".

A lo largo del documento y para el desarrollo de este experimento se estará trabajando con el software Code Composer Studio para el desarrollo del experimento 11.6. El experimento en cuestión tiene como objetivo crear un programa en lenguaje C que permita procesar una huella dactilar con la finalidad de comprender el procesamiento digital de imágenes. La imagen de huella dactilar utilizada para este experimento es un archivo BMP estándar para simplificar. Además, este experimento utiliza sólo los píxeles verdes, es decir, los componentes G de la imagen RGB.

Texas Instruments provee un software llamado Code Composer Studio (CCS) que es un entorno de desarrollo diseñado para programar las diferentes familias de dispositivos que fabrica. Este software se puede instalar en diferentes sistemas operativos, tales como Windows, Linux y Mac OS. El entorno de Code Composer Studio permite desarrollar programas teóricamente **"sin límite de tamaño" para todos los dispositivos MSP430 Value Line** disponibles actualmente. [1]

Para el desarrollo del programa de filtrado de imágenes, se implementará lenguaje de programación C. El lenguaje de programación C fue creado por Brian Kernighan y Dennis Ritchie a mediados de los años 70. El lenguaje C es un lenguaje para programadores en el sentido de que proporciona una gran flexibilidad de programación y una muy baja comprobación de incorrecciones, de forma que el lenguaje deja bajo la responsabilidad del programador acciones que otros lenguajes realizan por sí mismos. El lenguaje C es un lenguaje estructurado, pero no es estructurado por bloques, o sea, no es posible declarar subrutinas (pequeños trozos de programa) dentro de otras subrutinas, a diferencia de como sucede con otros lenguajes estructurados tales como el Pascal. Además, el lenguaje C no es rígido en la comprobación de tipos de datos, permitiendo fácilmente la conversión entre diferentes tipos de datos y la asignación entre tipos de datos diferentes. Todo programa de C consta, básicamente, de un conjunto de funciones, y una función llamada main, la cual es la primera que se ejecuta al comenzar el programa, llamándose desde ella al resto de funciones que compongan nuestro programa. [2]

Un archivo BMP es un formato de imagen que utiliza por defecto el sistema operativo Windows para dar independencia en funcionalidad respecto a los dispositivos de visualización. El sitio de soporte de Microsoft nos explica que BMP es el formato que Windows usa de manera estándar para las imágenes independientes del dispositivo o aplicación. Un archivo con la extensión .bmp se conoce como un archivo de imagen de mapa de bits o Windows bitmap. [3]

3. DESARROLLO

Las aplicaciones de huellas dactilares utilizan varios algoritmos de procesamiento de imágenes como el filtrado de imágenes para mejorar los bordes. Este experimento presenta una aplicación simplificada de huellas dactilares utilizando el C5505 eZdsp. Para las aplicaciones de huellas dactilares en el mundo real.

Para las aplicaciones de huellas dactilares del mundo real, existen varios requisitos para la adquisición de huellas dactilares. Sin embargo, la imagen de huella dactilar utilizada para este experimento es un archivo BMP estándar para simplificar. Además, este experimento utiliza sólo los píxeles verdes, es decir, los componentes G de la imagen RGB.

El experimento fue desarrollado en el software Code Composer, junto con la implementación de la tarjeta Texas Instruments. Así mismo, el experimento fue estructurado de acuerdo con las instrucciones del experimento 11.6 indicado.

Comenzando con la apertura de nuestro experimento 11.6 teniendo así todos los archivos indispensables para el desarrollo de este.

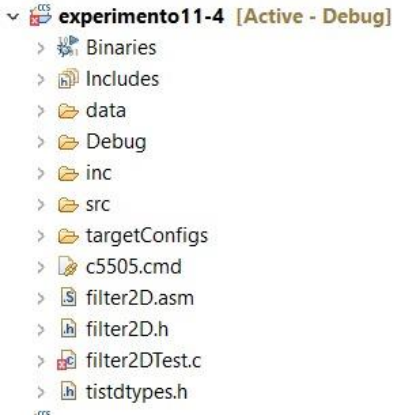


Imagen 1. "Archivos de proyecto 11.4"

Los procedimientos del experimento se enumeran como sigue:

1. Importar el proyecto CCS desde el paquete de software complementario y reconstruya el proyecto.
2. Cargar el programa en el eZdsp y ejecute el experimento para obtener la huella digital resultante. Comparar las imágenes de salida con la huella digital original como se muestra en la Observar las siguientes diferencias:
   (a) Mejora del contraste de la imagen.
   (b) Mejora en los bordes.
3. Perfilar el programa de huellas dactilares para identificar las cinco funciones clave que consumen la mayor parte de la potencia de procesamiento. Optimizar las funciones utilizando los intrínsecos del C55xx. Una vez que el programa haya sido modificado y probado completamente, ejecutar el experimento para perfilar el ahorro computacional logrado por el uso de intrínsecos.

3.1 CODIGO

Dentro de este programa contamos con diferentes opciones respectivamente, los cuales deben ser programados para verse visualizados en el menú, además de ser funciones que son llamadas para el funcionamiento del programa.

```
// Segement
printf("\t1. generate Segement BMP image\n");
TA_Segement(pImg);                                          // Segement
fpOut = fopen("..\\data\\1_Segement.bmp", "wb");            // Open Segement file for write
writeFile(fpOut, temp, pImg, bmpHeader, offset, 1);
fclose(fpOut);

// Gray Stretch
printf("\t2. generate Gray Stretch BMP image\n");
TA_Graystretch(pImg);                                       // Gray Stretch
fpOut = fopen("..\\data\\2_Graystretch.bmp", "wb");         // Open Gray Stretch file for write
writeFile(fpOut, temp, pImg, bmpHeader, offset, 2);
fclose(fpOut);

// Block Direction
printf("\t3. generate Block Direction BMP image\n");
TA_GetBlockDirection(pImg, (Int16 *)pVbin, (Int16 *)pTemp); // Get Block Direction

fpOut = fopen("..\\data\\3_BlockDirection.bmp", "wb");      // Open Block Direction file for write
writeFile(fpOut, temp, pVbin, bmpHeader, offset, 3);
fclose(fpOut);

// Gabor Filter
printf("\t4. generate Gabor Filter BMP image\n");
TA_gabor_filter(pImg,(short *)pVbin);                       // Gabor Filtering
fpOut = fopen("..\\data\\4_GaborFilter.bmp", "wb");         // Open Gabor Filter file for write
writeFile(fpOut, temp, pVbin, bmpHeader, offset, 4);
fclose(fpOut);

// Binarization
printf("\t5. generate Binarization BMP image\n");
TA_BinImage(pVbin);                                         // Binarization Gray Image
fpOut = fopen("..\\data\\5_Binarization.bmp", "wb");        // Open Binarization file for write
writeFile(fpOut, temp, pVbin, bmpHeader, offset, 5);
fclose(fpOut);

// Thinning
printf("\t6. generate Thinning BMP image\n");
TA_Thin_image(pVbin,pImg,pTemp);                            // Thinning Binary Image
fpOut = fopen("..\\data\\6_Thinning.bmp", "wb");            // Open Thinning file for write
writeFile(fpOut, temp, pVbin, bmpHeader, offset, 6);
fclose(fpOut);

printf("Exp --- completed\n");
```

Imagen 2. Código de los procesos.

Por otra parte, tenemos la optimización de cada una de las funciones de los procesos utilizando los intrínsecos del C55xx para perfilar el ahorro computacional

```
void TA_Segement(Uint16 *pImg)
{
#if USE_FDK_DEMO_CODE
    This program is part of Texas Instruments TMS320C5515 Fingerprint Development Kit
    (http://processors.wiki.ti.com/index.php/C5000_Fingerprint_Kit).
#endif
}
```

Imagen 3. Optimización del proceso segment.

```
void TA_Graystretch(Uint16 *pImg)
{
#if USE_FDK_DEMO_CODE
     This program is part of Texas Instruments TMS320C5515 Fingerprint Development Kit
     (http://processors.wiki.ti.com/index.php/C5000_Fingerprint_Kit).
#endif
}
```

Imagen 4. Optimización del proceso GrayStretch

```
void TA_GetBlockDirection(Uint16* Img,Int16* Gx,Int16* Gy)
{
#if USE_FDK_DEMO_CODE
     This program is part of Texas Instruments TMS320C5515 Fingerprint Development Kit
     (http://processors.wiki.ti.com/index.php/C5000_Fingerprint_Kit).
#endif
}
```

Imagen 5. Optimización del proceso Get Block Direction

```
void TA_gabor_filter(Uint16 *Img, Int16 *enhance_image)
{
#if USE_FDK_DEMO_CODE
     This program is part of Texas Instruments TMS320C5515 Fingerprint Development Kit
     (http://processors.wiki.ti.com/index.php/C5000_Fingerprint_Kit).
#endif
}
```

Imagen 6. Optimización del proceso Gabor filter.

```
void TA_BinImage (Uint16 *image)
{
#if USE_FDK_DEMO_CODE
     This program is part of Texas Instruments TMS320C5515 Fingerprint Development Kit
     (http://processors.wiki.ti.com/index.php/C5000_Fingerprint_Kit).
#endif
}
```

Imagen 7. Optimización del proceso de Binarización

```
void TA_Thin_image(Uint16 *Img,Uint16 *Temp,Uint16 *T_mark)
{
#if USE_FDK_DEMO_CODE
     This program is part of Texas Instruments TMS320C5515 Fingerprint Development Kit
     (http://processors.wiki.ti.com/index.php/C5000_Fingerprint_Kit).
#endif
}
```

Imagen 8. Optimización del proceso Thinning

3.2 PROGRAMA EJECUTADO

Una vez desarrollado el programa y ya con los archivos completos, se procede a ejecutar el programa, por lo que en primera instancia observamos el menú del proceso.

```
Exp --- finger print experiment
        1. generate Segement BMP image
        2. generate Gray Stretch BMP image
        3. generate Block Direction BMP image
        4. generate Gabor Filter BMP image
        5. generate Binarization BMP image
        6. generate Thinning BMP image
Exp --- completed
```

Imagen 9. Meno de procesos.

Finalmente, podemos confirmar el funcionamiento del programa visualizando la carpeta donde se encuentran los archivos del programa, la cual previamente ya contaba con imágenes de huellas dactilares y que fueron cargadas al programa.

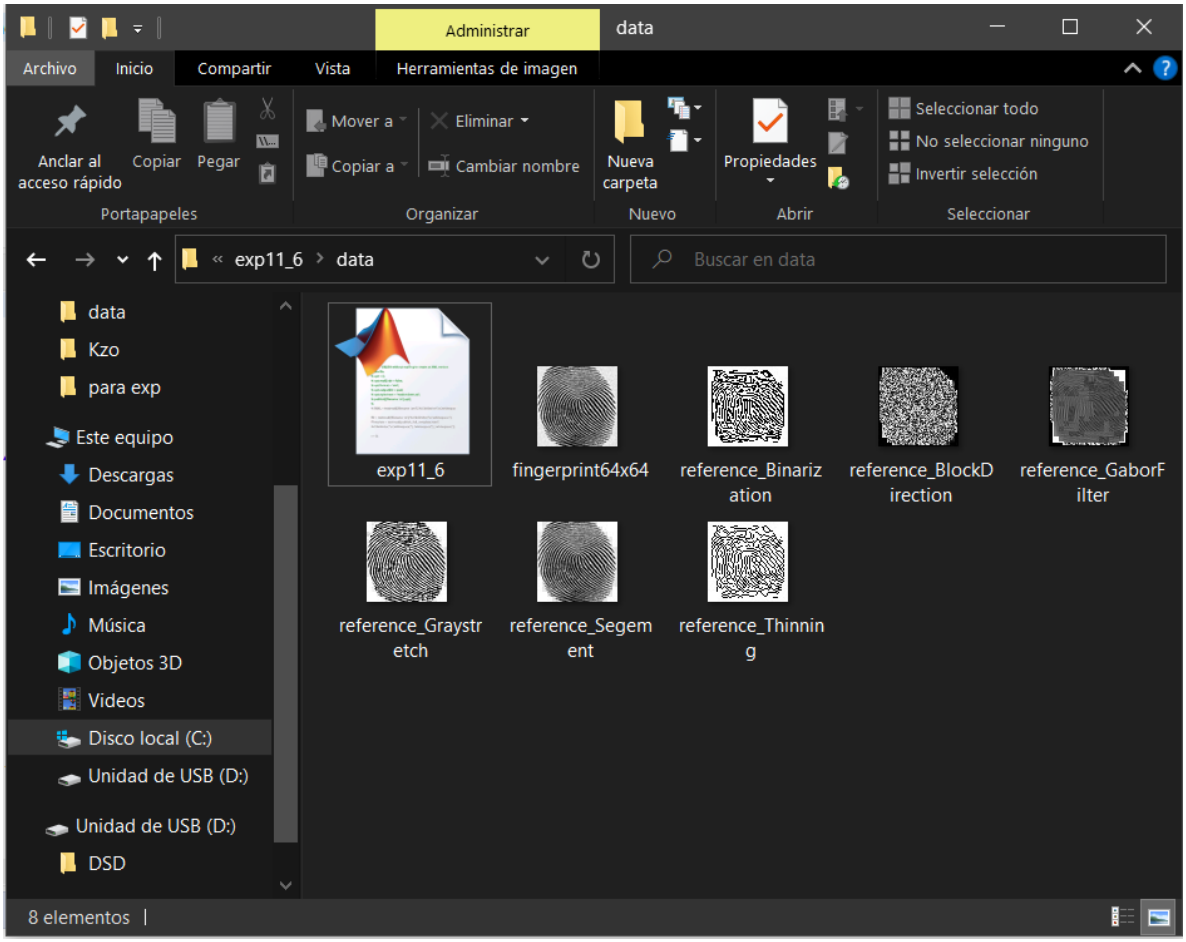


Imagen 10. Data inicial

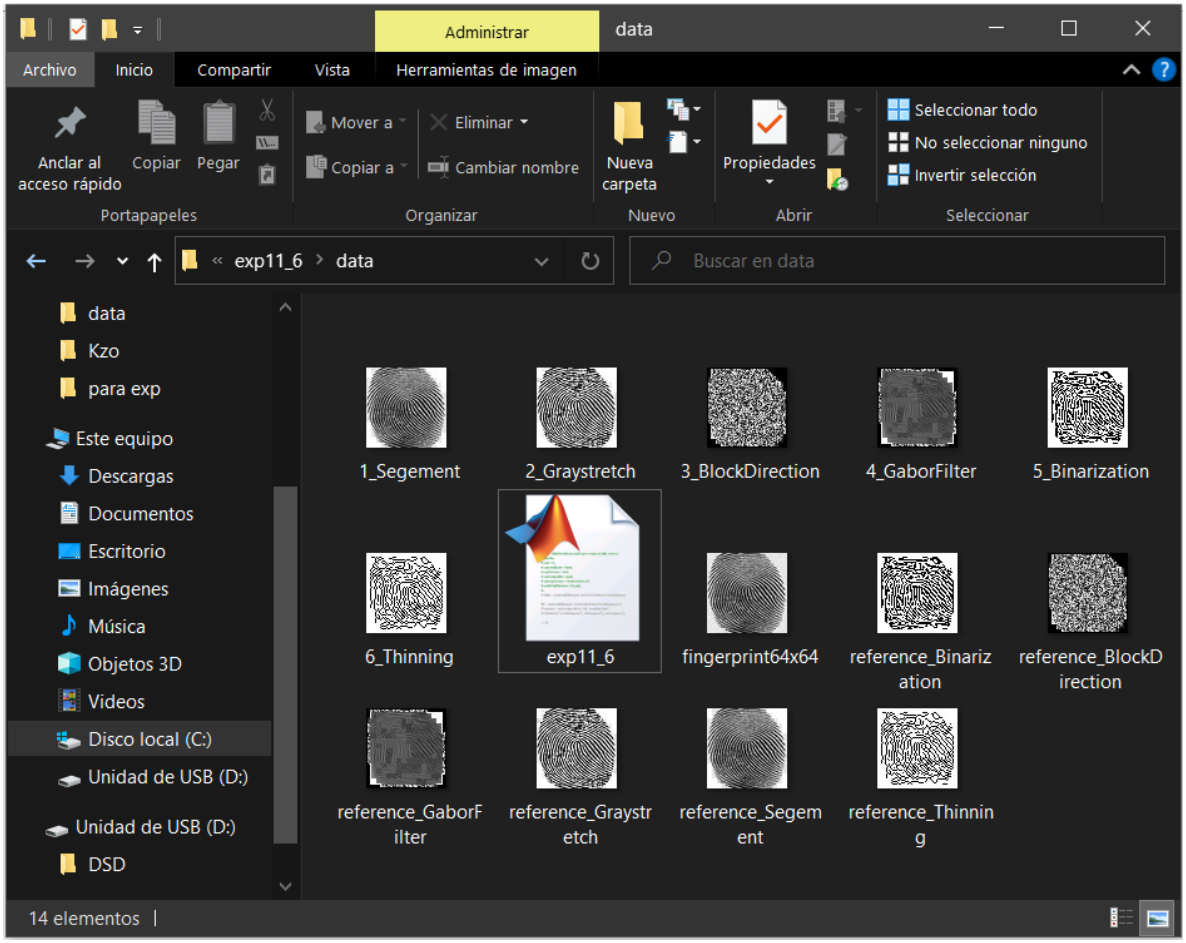


Imagen 11. Data final después de la ejecución del programa.

4. CONCLUSIONES

- **ALABAT MONTOYA MARIA FERNANDA**

- **BRIONES CRUZ JUAN CARLOS**

- **SALAS ORTIZ ESTEPHANI**

- **VAZQUEZ RODRIGUEZ ULISES**

5. REFERENCIAS

[1] Control Suit Man Capitulo 2. Code Composer Studio. http://odin.fi-b.unam.mx/labdsp/files/ADSP/2018-2/apuntes/Control_Suit_Man_C2t.pdf

[2] Bonet E. (S.F). “Lenguaje en C”. [PDF]. Disponible en: https://informatica.uv.es/estguia/ATD/apuntes/laboratorio/Lenguaje-C.pdf
[3] E. Mac. "¿Cuál es el significado de BMP?" Techlandia. https://techlandia.com/significado-bmp-hechos_103910/ (accedido el 7 de junio de 2022).